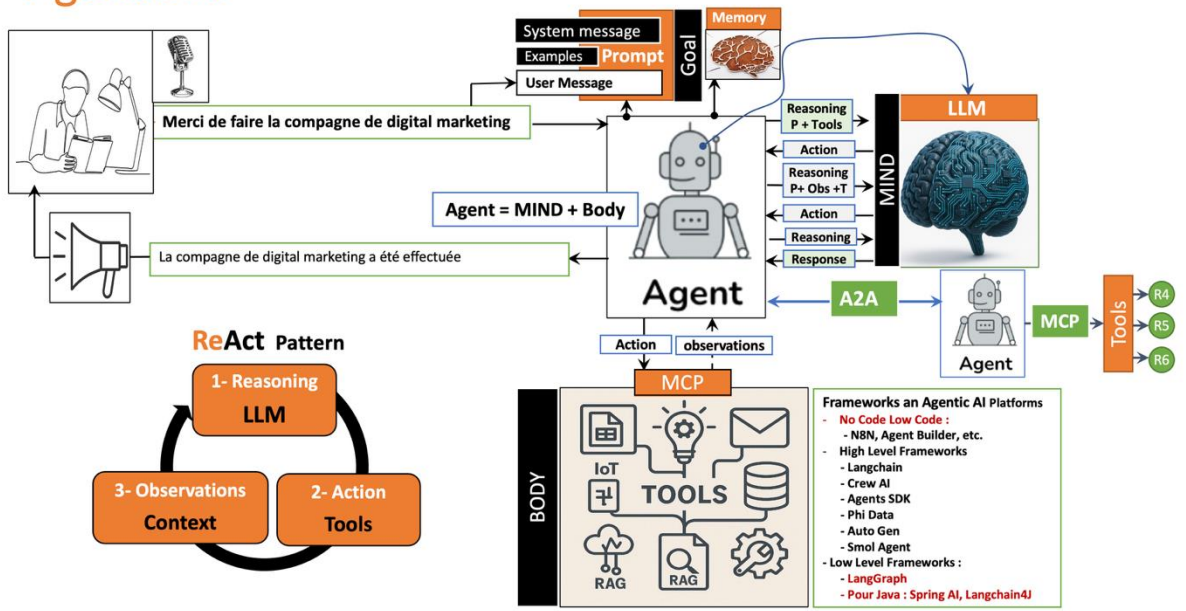
RAG Multi Modal Agentique

Mohamed Youssfi, Enseignant Chercheur, ENSET Mohammedia, Directeur Laboratoire Informatique, Intelligence Artificielle et Cyber Sécurité



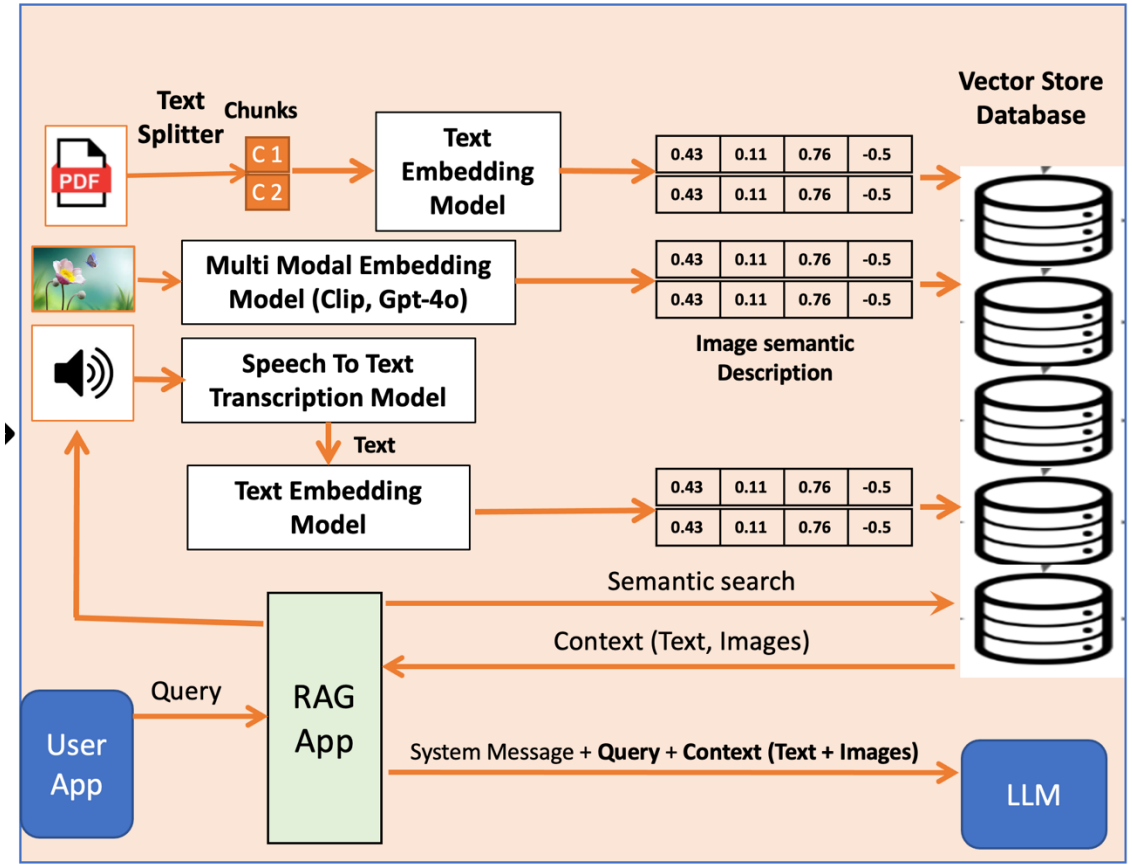
Agentic AI



QWEN-3

Gemma 4

Chroma

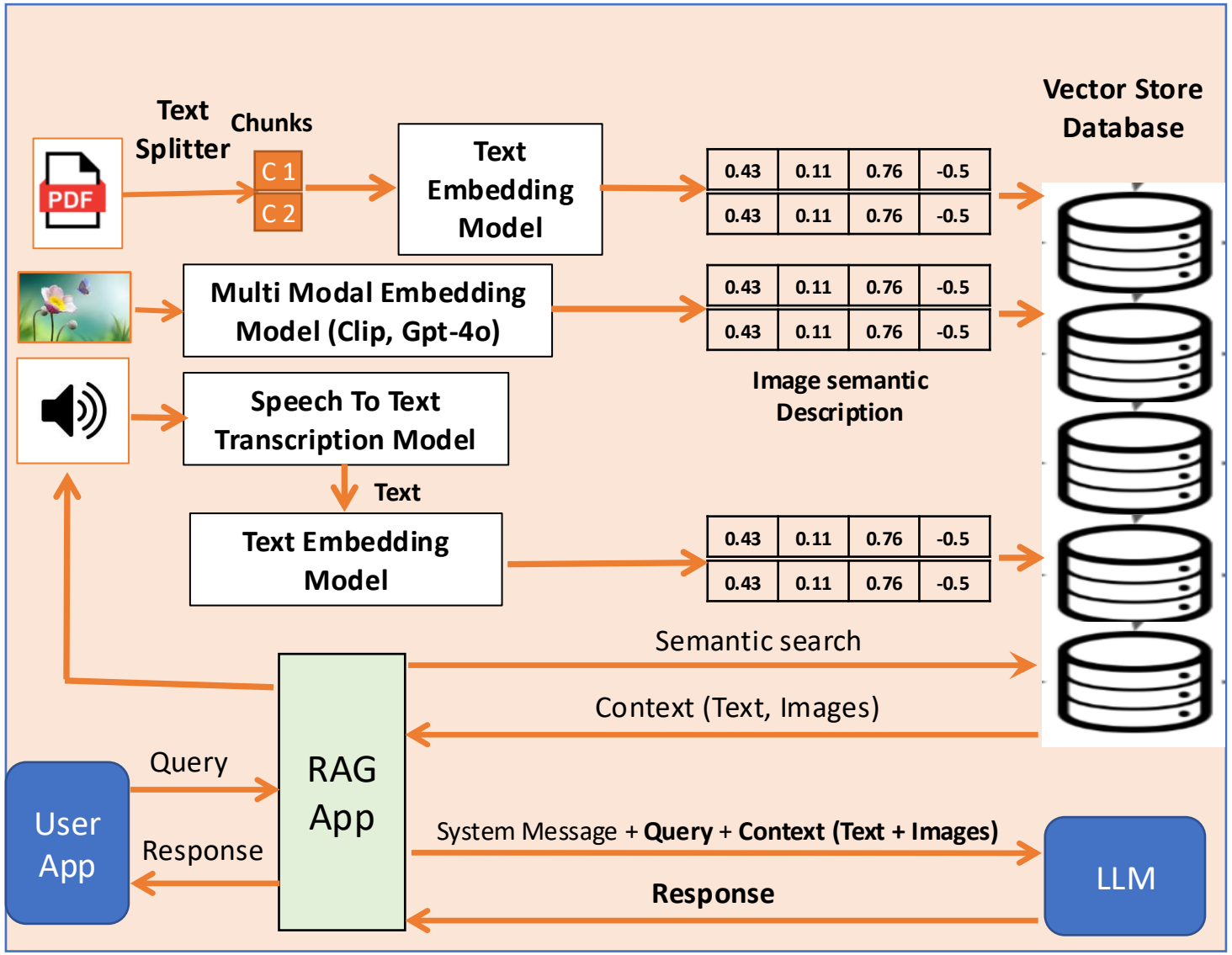


Multi Modal Retrieval Augmented Generation

Multi Modal RAG



Multi RAG Retrieval Augmented Generation

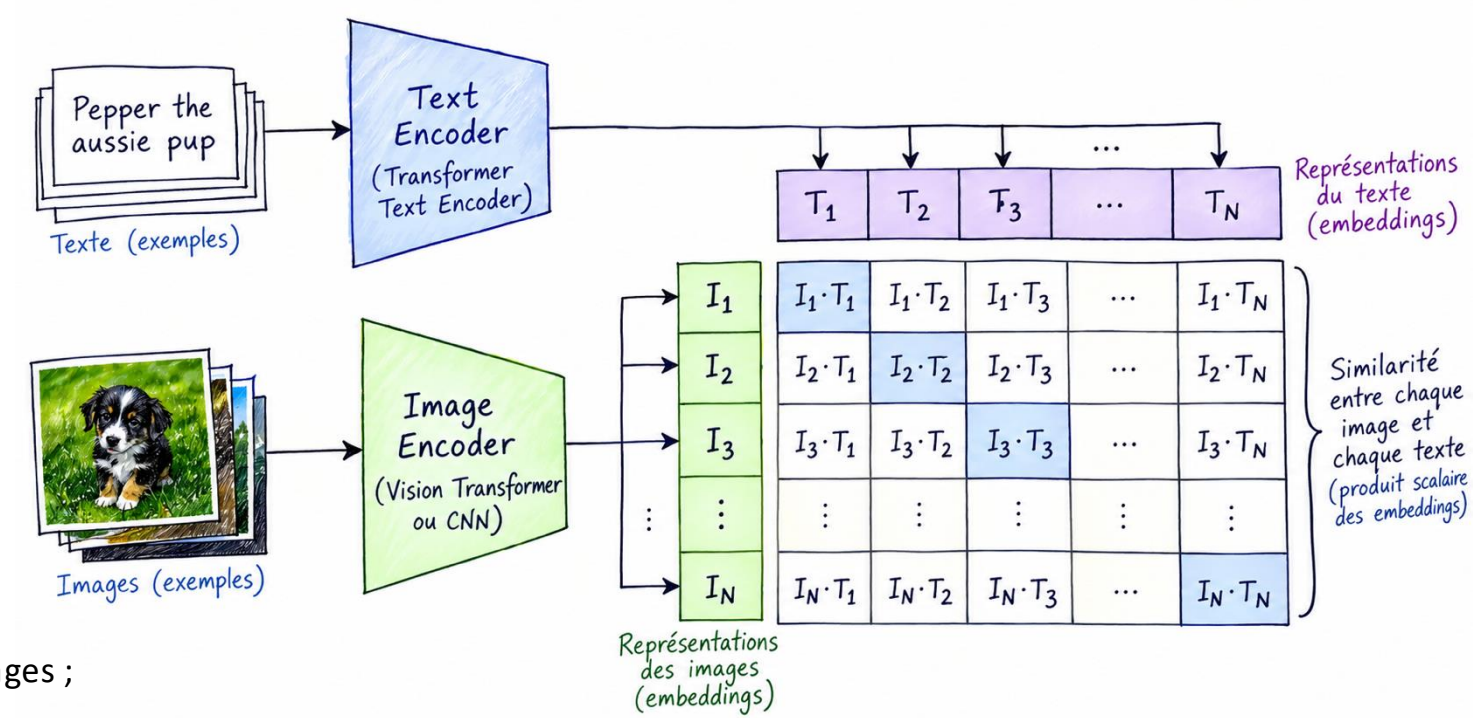


Modèle CLIP

- Contrastive Language–Image Pre-training — OpenAI, 2021
- Apprendre à représenter **images et textes dans un espace latent commun**, où une image est proche de sa description.

- **Applications**
  - **Classification zero-shot** : Reconnaître une catégorie à partir de descriptions, sans entraînement spécifique à la tâche.
  - **Recherche multimodale** :
    - Text to Text
    - Text to Image
    - Image ti Text
    - Image to Image

MODEL_NAME = **"ViT-B-32"**
PRETRAINED = **"laion2b_s34b_b79k"**



- **ViT** : architecture *Vision Transformer* pour l'encodeur d'images ;
  - **B** : variante *Base* ;
  - **32** : taille des patchs d'image, soit **32 × 32 pixels**.
- Les poids **laion2b_s34b_b79k** proviennent d'un entraînement sur des paires image–texte issues de LAION.
  - **2b**: Environ **2 milliards de paires image–texte** dans le jeu de données utilisé.
  - **s34b** : Environ **34 milliards de paires présentées** au modèle au cours de l'entraînement, en comptant les répétitions. Le **s signifie samples** : exemples.
  - **b79k** : Environ **79 000 paires traitées** par étape d'entraînement, au total sur l'ensemble des GPU. Le premier **b signifie batch** : lot.

Modèle CLIP

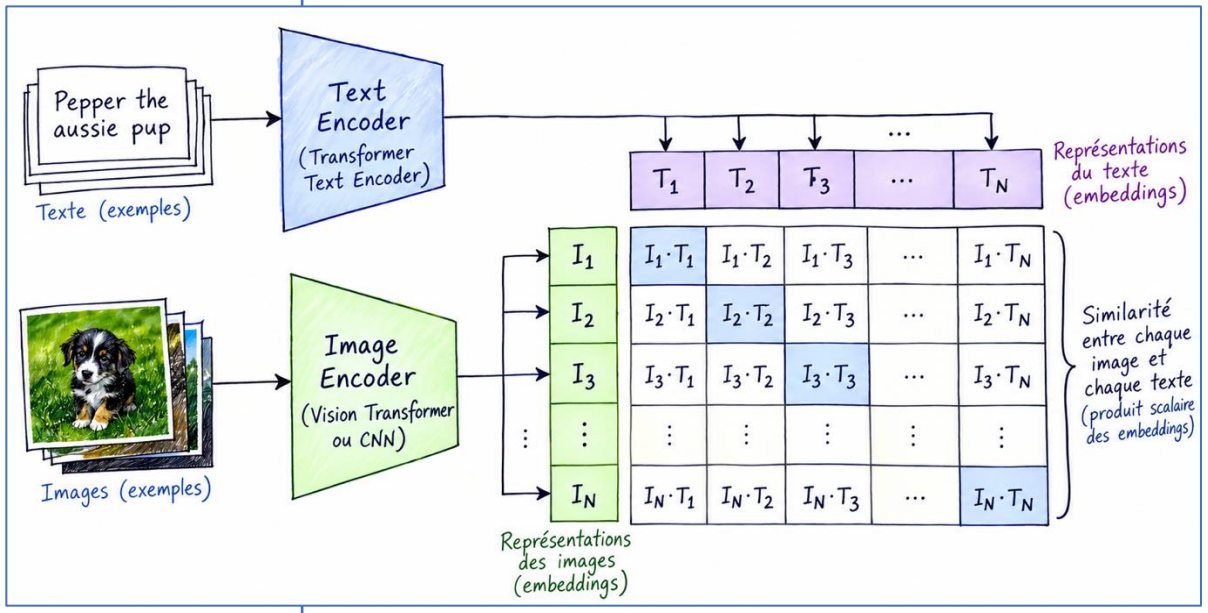
```
import torch
import open_clip
from PIL import Image
DEVICE = "cuda" if torch.cuda.is_available() else "cpu"
```

```
MODEL_NAME = "ViT-B-32"
PRETRAINED = "laion2b_s34b_b79k"
# Charger le modèle et le prétraitement des images
model, _, preprocess = open_clip.create_model_and_transforms(
    MODEL_NAME,
    pretrained=PRETRAINED
)
model = model.to(DEVICE).eval()
tokenizer = open_clip.get_tokenizer(MODEL_NAME)

# --- Préparer le texte ---
texte = "Un chat sur un canapé"
tokens = tokenizer([texte]).to(DEVICE)

# --- Préparer l'image ---
with Image.open("architecture_transformer.png") as image:
    tenseur_image = preprocess(image.convert("RGB"))

# Ajouter la dimension batch : [3, H, W] → [1, 3, H, W]
tenseur_image = tenseur_image.unsqueeze(0).to(DEVICE)
```

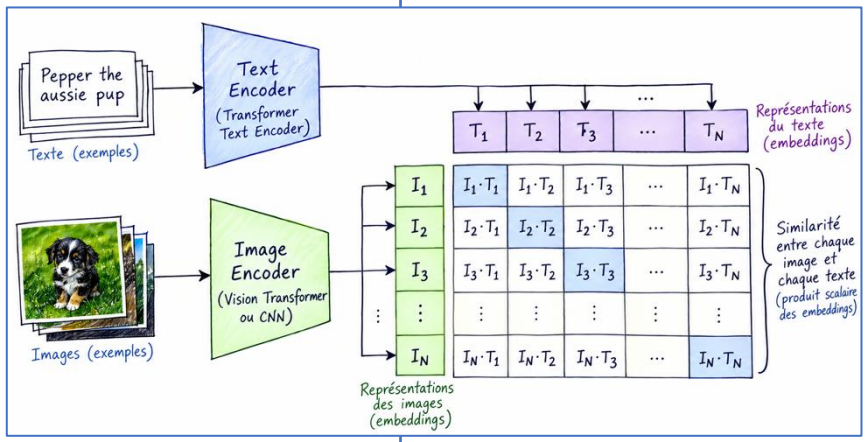


Modèle CLIP

```python
# --- Encoder le texte et l'image ---
with torch.inference_mode():
    vecteur_texte = model.encode_text(tokens)
    vecteur_image = model.encode_image(tenseur_image)
    # Normaliser les embeddings
    vecteur_texte = vecteur_texte / vecteur_texte.norm(dim=-1, keepdim=True)
    vecteur_image = vecteur_image / vecteur_image.norm(dim=-1, keepdim=True)
# Convertir en listes Python utilisables par ChromaDB
vecteur_texte = vecteur_texte[0].cpu().tolist()
vecteur_image = vecteur_image[0].cpu().tolist()


print("Dimension texte :", len(vecteur_texte))  # 512
print("Dimension image :", len(vecteur_image))  # 512


print("Embedding texte (5 premières valeurs) :", vecteur_texte[:5])
print("Embedding image (5 premières valeurs) :", vecteur_image[:5])
```

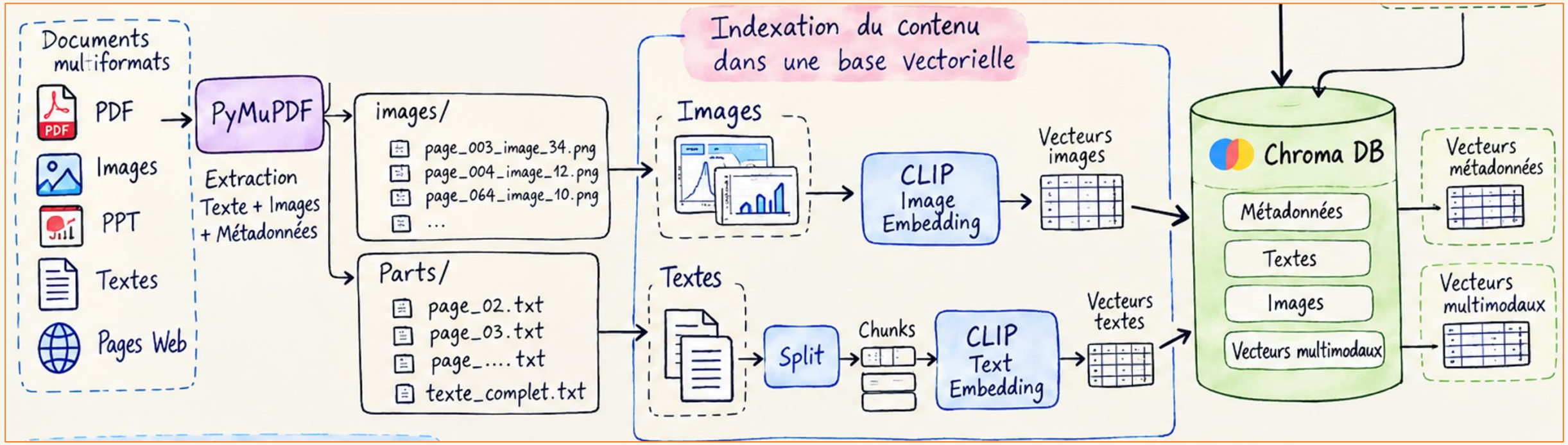



```
Dimension texte : 512
Dimension image : 512
Embedding texte (5 premières valeurs) : [0.02338840253651142, 0.01853043958544731, -0.0027200
Embedding image (5 premières valeurs) : [0.03023669123649597, -0.13937945663928986, -0.002614
```

RAG Multi Modal

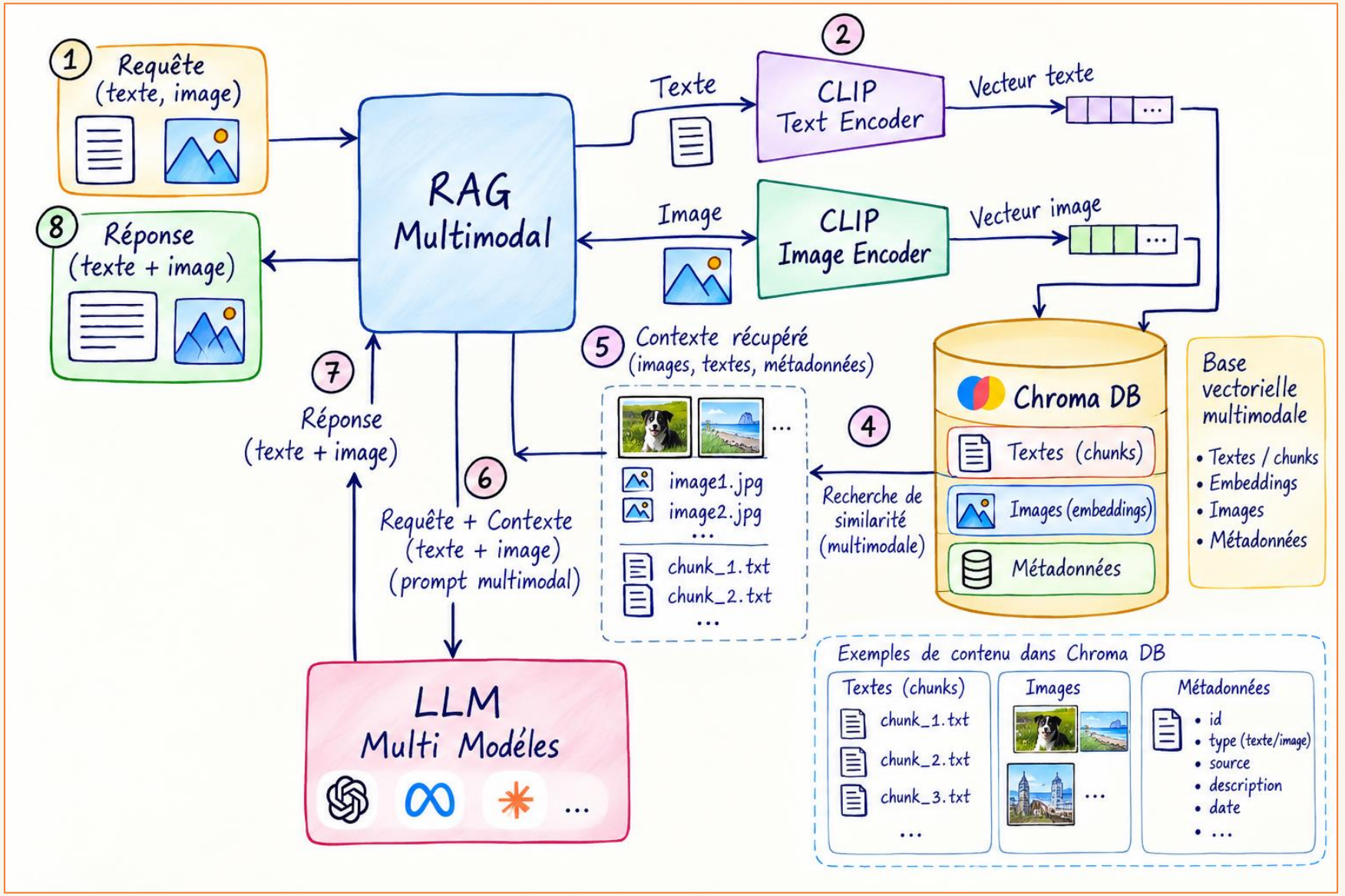
1- Extraction des textes et des images

2- Indexation du contenu dans la base vectorielle



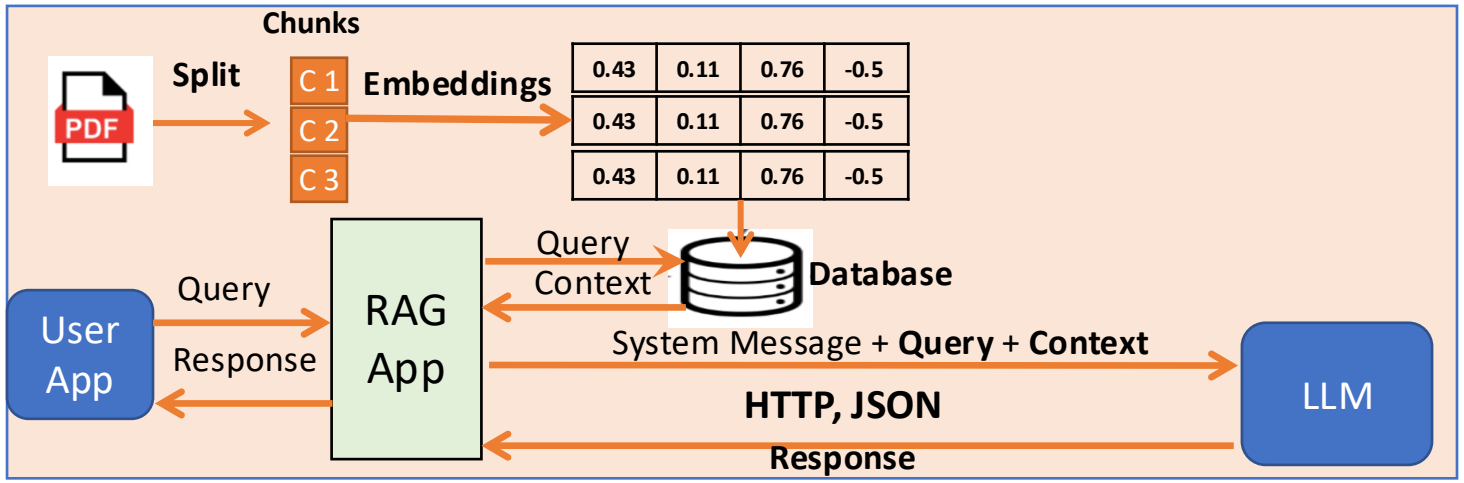
RAG Multi Modal

3- La partie Retrival & Generation

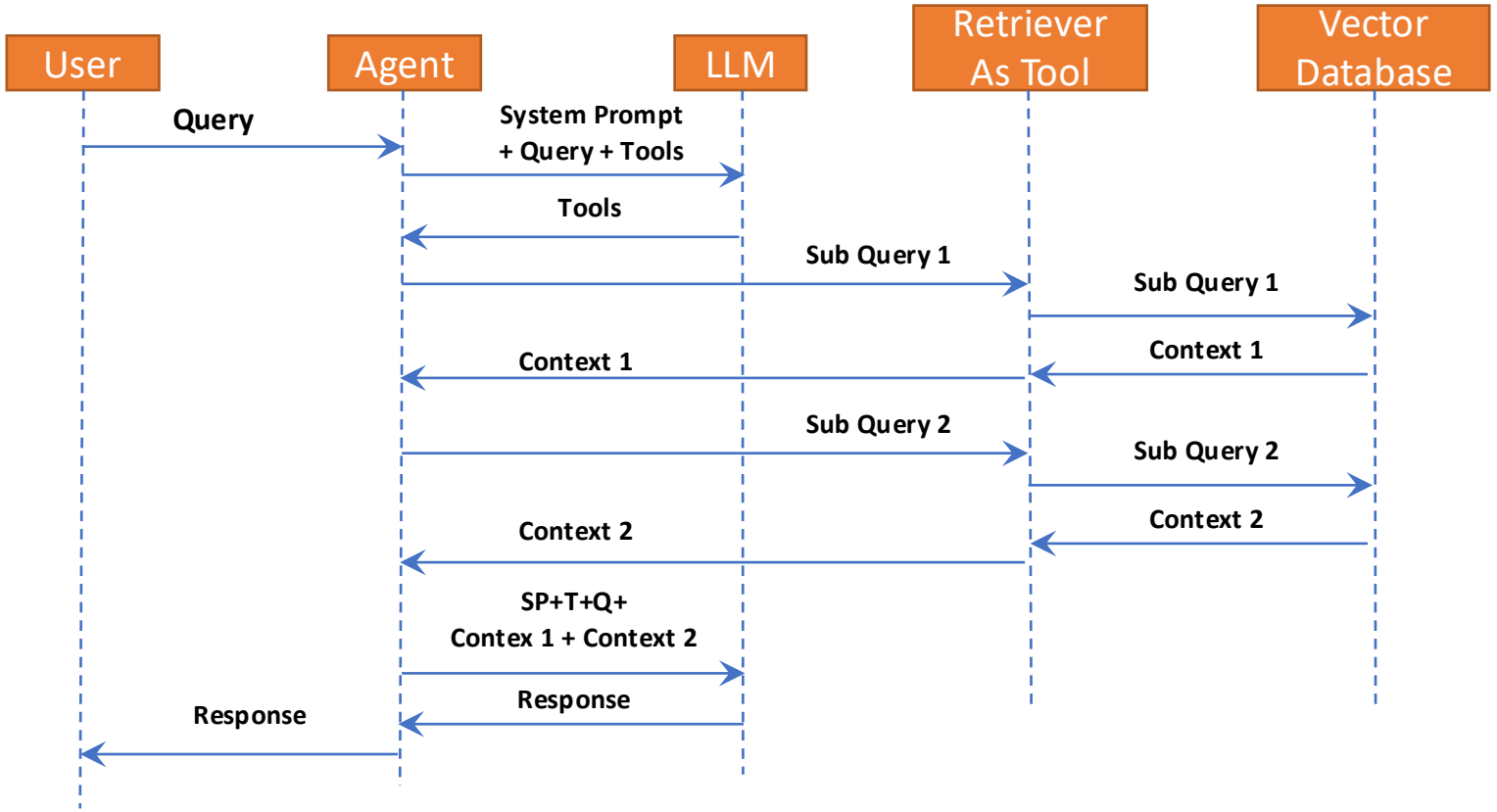


Agentic RAG

Classic RAG

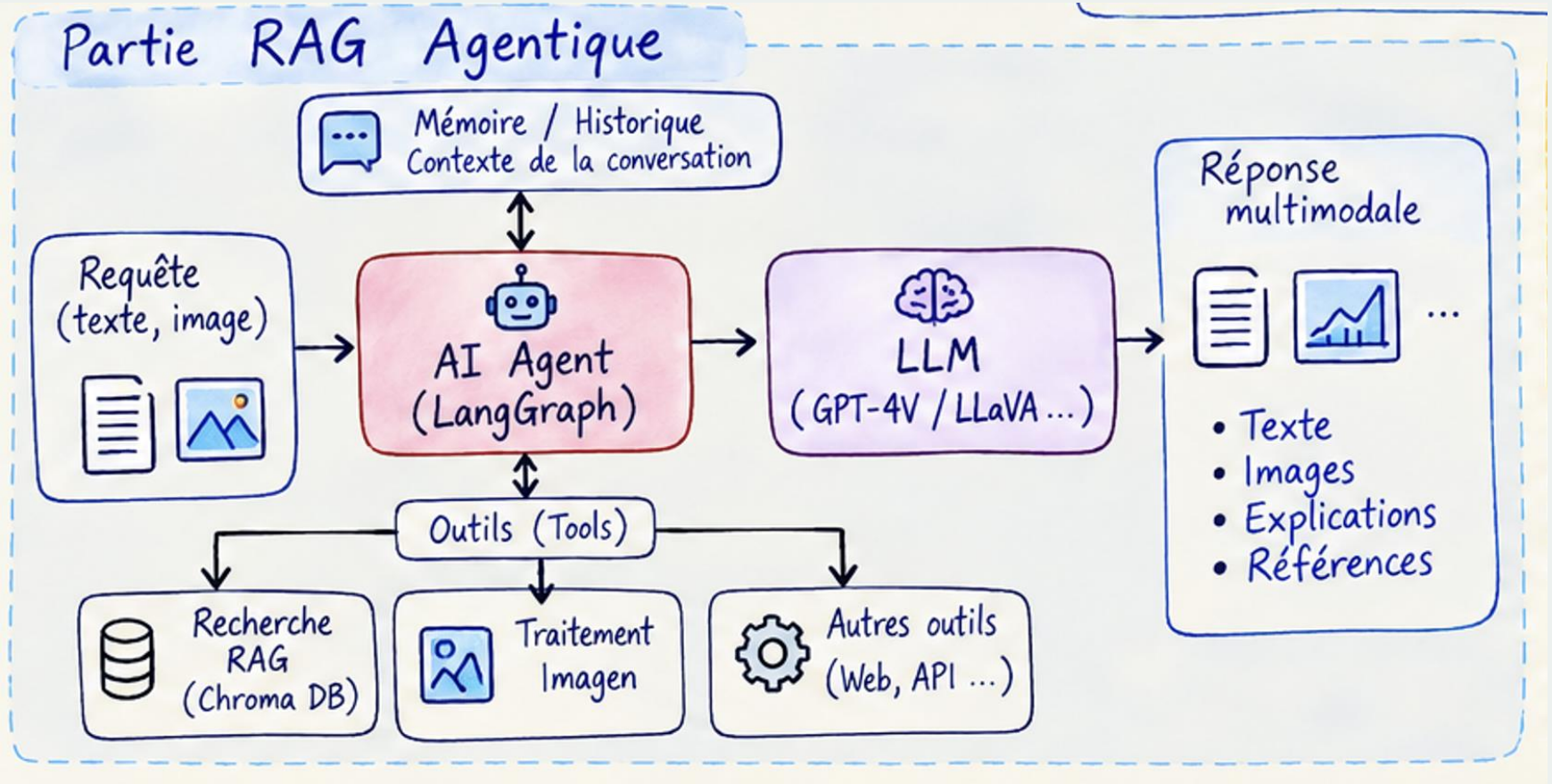


Agentic RAG

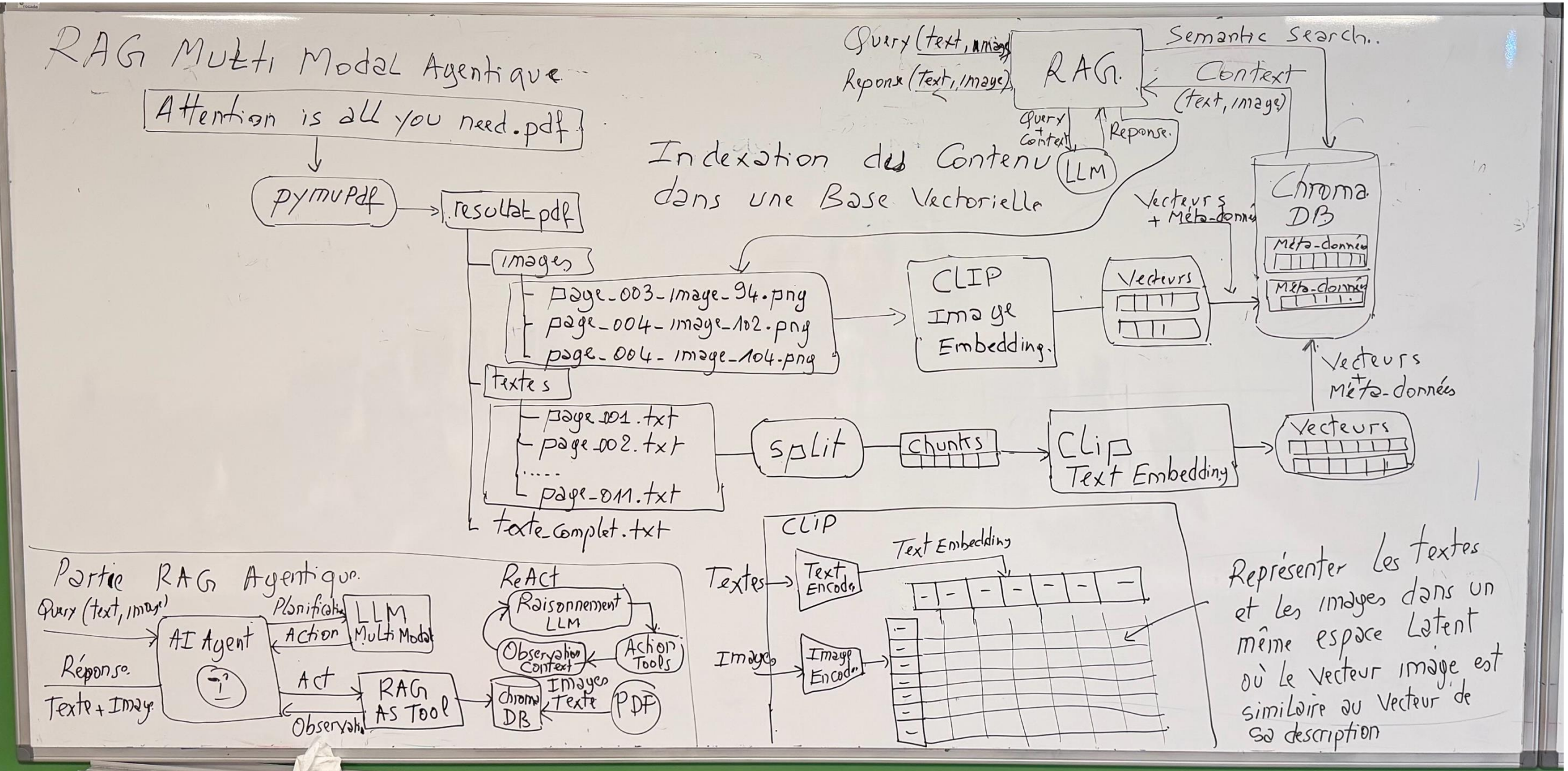


RAG Multi Modal Agentique

4- Partie Agentique

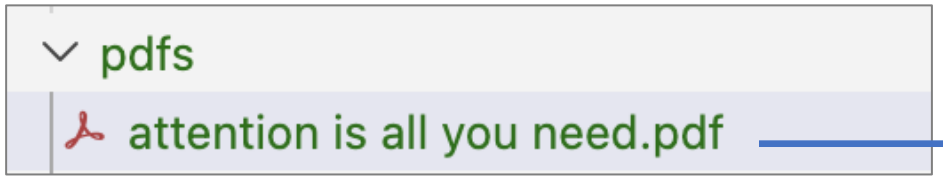


RAG Multi Modal Agentique



RAG Multi Modal

Source PDF




```python
from pathlib import Path
import pymupdf

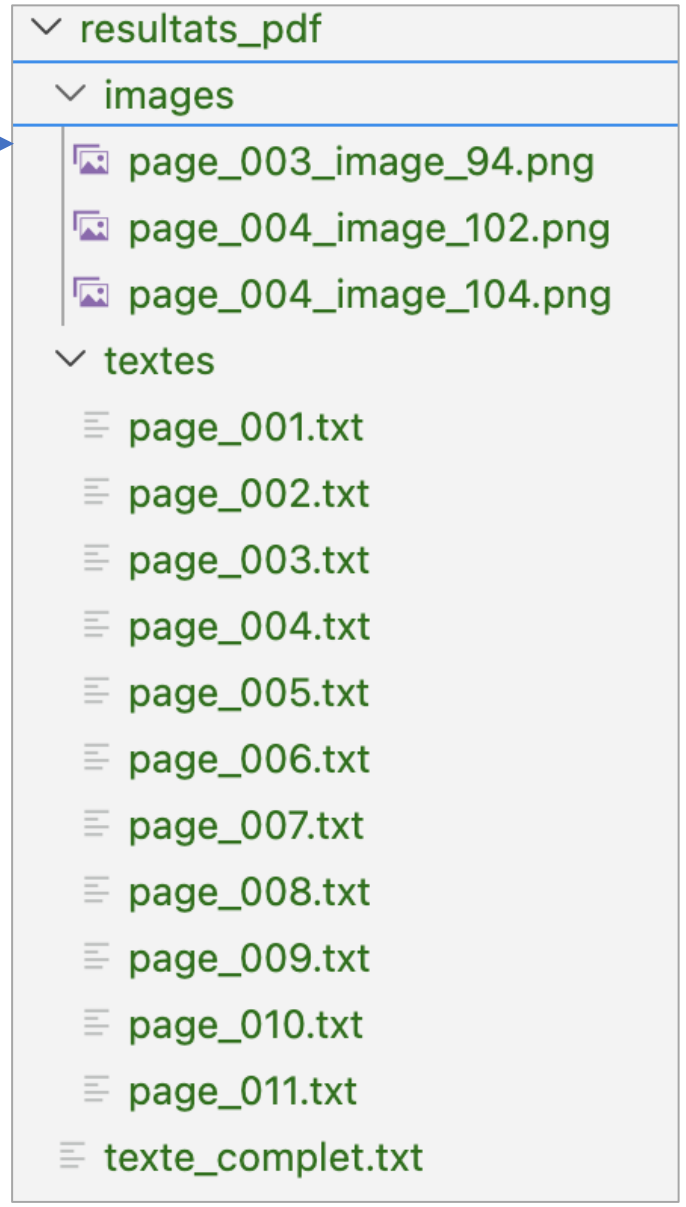
PDF_PATH = Path("./pdfs/attention is all you need.pdf")

OUTPUT_DIR = Path("resultats_pdf")
TEXT_DIR = OUTPUT_DIR / "textes"
IMAGE_DIR = OUTPUT_DIR / "images"

if not PDF_PATH.is_file():
    raise FileNotFoundError(f"Fichier introuvable : {PDF_PATH.resolve()}")

TEXT_DIR.mkdir(parents=True, exist_ok=True)
IMAGE_DIR.mkdir(parents=True, exist_ok=True)

print(f"PDF : {PDF_PATH.resolve()}")
print(f"Résultats : {OUTPUT_DIR.resolve()}")
```



RAG Multi Modal

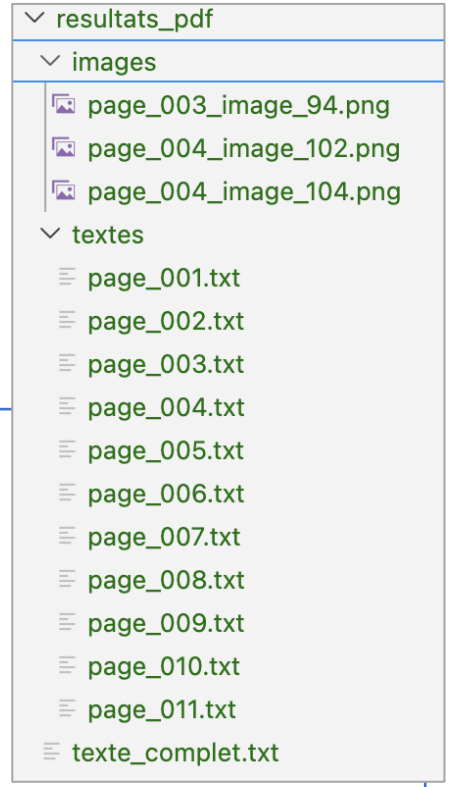
Extraction des images et des textes

```
textes = []
nombre_images = 0
pages_sans_texte = []
with pymupdf.open(PDF_PATH) as document:
    nombre_pages = len(document)
    for numero_page, page in enumerate(document, start=1):
        texte = page.get_text("text", sort=True)
        fichier_texte = TEXT_DIR / f"page_{numero_page:03d}.txt"
        fichier_texte.write_text(texte, encoding="utf-8")
        textes.append(f"===== PAGE {numero_page} =====\n\n{texte}")
        if not texte.strip():
            pages_sans_texte.append(numero_page)
        images_vues = set()
        for image_info in page.get_images(full=True):
            xref = image_info[0]
            if xref in images_vues:
                continue
            images_vues.add(xref)
            try:
                image = document.extract_image(xref)
                if not image:
                    print(f"Image non extractible")
                    continue
                extension = image["ext"]
                fichier_image = (
                    IMAGE_DIR
                    / f"page_{numero_page:03d}_image_{xref}.{extension}"
                )
                fichier_image.write_bytes(image["image"])
                nombre_images += 1
            except Exception as erreur:
                print(
                    f"Erreur d'extraction : page {numero_page}, "
                    f"image {xref} : {erreur}"
                )
```

```
# Enregistrer le texte de tout le document.
(OUTPUT_DIR / "texte_complet.txt").write_text(
    "\n\n".join(textes),
    encoding="utf-8"
)

print(f"Pages traitées : {nombre_pages}")
print(f"Images enregistrées : {nombre_images}")

if pages_sans_texte:
    print("Pages sans texte détectable :", pages_sans_texte)
    print("Si elles contiennent du texte scanné, utilisez l'OCR ")
```



RAG Multi Modal

Afficher un aperçu des résultats

```
from PIL import Image
from IPython.display import display

texte_complet = (OUTPUT_DIR / "texte_complet.txt").read_text(
    encoding="utf-8"
)

print("APERÇU DU TEXTE\n")
print(texte_complet[:2000])

print("\nAPERÇU DES IMAGES\n")

for fichier in sorted(IMAGE_DIR.iterdir())[:5]:
    print(fichier.name)

    with Image.open(fichier) as image:
        apercu = image.copy()
        apercu.thumbnail((600, 600))
        display(apercu)
```

```
APERÇU DU TEXTE

===== PAGE 1 =====

                Attention Is All You Need



    Ashish Vaswani*      Noam Shazeer*       Niki Parmar*      Jakob Uszkoreit*
      Google Brain         Google Brain       Google Research     Google Research
avaswani@google.com   noam@google.com   nikip@google.com   usz@google.com

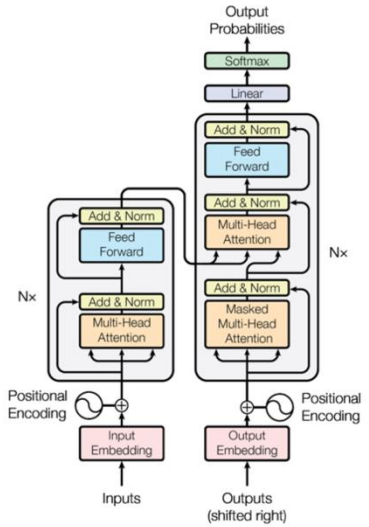
       Llion Jones*          Aidan N. Gomez*†            Łukasz Kaiser*
     Google Research         University of Toronto            Google Brain
    llion@google.com     aidan@cs.toronto.edu     lukaszkaiser@google.com

                                          Illia Polosukhin*‡
                        illia.polosukhin@gmail.com

...
```

```
APERÇU DES IMAGES

page_003_image_94.png
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
```




```
page_004_image_102.png
```



RAG Multi Modal

Charger le modèle et ouvrir ChromaDB

```
from pathlib import Path
import hashlib
import re
import torch
import open_clip
import chromadb
from PIL import Image
from open_clip.tokenizer import SimpleTokenizer

# Dossiers produits par les cellules d'extraction
OUTPUT_DIR = Path("resultats_pdf")
TEXT_DIR = OUTPUT_DIR / "textes"
IMAGE_DIR = OUTPUT_DIR / "images"

DEVICE = "cuda" if torch.cuda.is_available() else "cpu"

MODEL_NAME = "ViT-B-32"
PRETRAINED = "laion2b_s34b_b79k"

model, _, preprocess = open_clip.create_model_and_transforms(
    MODEL_NAME,
    pretrained=PRETRAINED
)

model = model.to(DEVICE).eval()
tokenizer = open_clip.get_tokenizer(MODEL_NAME)
```

```
# Tokeniseur BPE adapté au modèle CLIP choisi
bpe = SimpleTokenizer()

# Base persistante sur disque
client = chromadb.PersistentClient(path="chroma_pdf")

collection = client.get_or_create_collection(
    name="pdf_clip_vit_b32",
    embedding_function=None,  # Les embeddings sont calculés par notre code
    metadata={"hnsw:space": "cosine"}
)

print("Appareil :", DEVICE)
print("Éléments déjà présents :", collection.count())
```

Chargement du modèle préentraîné

Le modèle choisi est **ViT-B-32** :

- **ViT** : architecture *Vision Transformer* pour l'encodeur d'images ;
  - **B** : variante *Base* ;
  - **32** : taille des patchs d'image, soit **32 × 32 pixels**.

Les poids **laion2b_s34b_b79k** proviennent d'un entraînement sur des paires image–texte issues de LAION.

- **2b**: Environ **2 milliards de paires image–texte** dans le jeu de données utilisé.
- **s34b** : Environ **34 milliards de paires présentées** au modèle au cours de l'entraînement, en comptant les répétitions. Le **s signifie samples** : exemples.
- **b79k** : Environ **79 000 paires traitées** par étape d'entraînement, au total sur l'ensemble des GPU. Le premier **b signifie batch** : lot.

RAG Multi Modal

Découpage du texte en fragments (chunks)

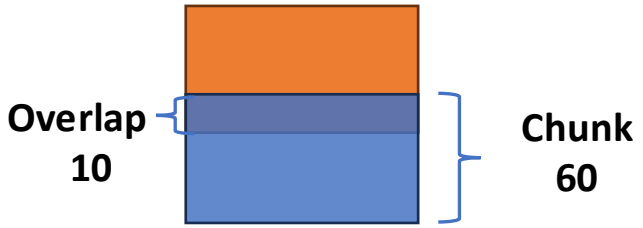
```python
def decouper_texte(texte, taille=60, chevauchement=10):
    """Découper le texte en fragments courts adaptés à CLIP."""
    if not 0 <= chevauchement < taille:
        raise ValueError("Le chevauchement doit être inférieur à la taille.")

    tokens = bpe.encode(texte)

    for debut in range(0, len(tokens), taille - chevauchement):
        fragment = bpe.decode(tokens[debut:debut + taille]).strip()

        if fragment:
            yield fragment

        if debut + taille >= len(tokens):
            break
```



Une autre façon de faire

```python
from langchain_text_splitters import RecursiveCharacterTextSplitter
def decouper_texte_V2(texte, taille=60, chevauchement=10):
    """Découper le texte en privilégiant les limites naturelles."""
    if not 0 <= chevauchement < taille:
        raise ValueError("Le chevauchement doit être inférieur à la taille.")
    splitter = RecursiveCharacterTextSplitter(
        chunk_size=taille,
        chunk_overlap=chevauchement,
        length_function=lambda texte: len(bpe.encode(texte)),
        separators=["\n\n", "\n", " ", ""],
        strip_whitespace=True,
    )
    yield from splitter.split_text(texte)
```

RAG Multi Modal

Encodage des textes en vecteurs avec CLIP

```
@torch.inference_mode()
def encoder_textes(textes):
    tokens = tokenizer(textes).to(DEVICE)
    vecteurs = model.encode_text(tokens).float()

    vecteurs = vecteurs / vecteurs.norm(
        dim=-1, keepdim=True
    ).clamp_min(1e-12)

    return vecteurs.cpu().tolist()
```

Textes → tokenisation → encodage CLIP → normalisation L2 → listes Python

Encodage des images en vecteurs avec CLIP

```
@torch.inference_mode()
def encoder_image(chemin):
    with Image.open(chemin) as image:
        tenseur = preprocess(image.convert("RGB"))

    tenseur = tenseur.unsqueeze(0).to(DEVICE)
    vecteur = model.encode_image(tenseur).float()

    vecteur = vecteur / vecteur.norm(
        dim=-1, keepdim=True
    ).clamp_min(1e-12)

    return vecteur.cpu().tolist()[0]
```

Pour la configuration utilisée, ces transformations comprennent généralement :

1. le redimensionnement ;
2. le recadrage central ;
3. la conversion en tenseur ;
4. la normalisation des valeurs des pixels.

Le tenseur obtenu possède généralement la forme **[3, 224, 224]**:

Dimension	Signification
3	Nombre de canaux de couleur
224	Hauteur de l'image préparée
224	Largeur de l'image préparée

Le recadrage peut retirer une partie des bords de l'image. La conversion en RGB supprime également un éventuel canal de transparence.

RAG Multi Modal

Indexation des textes du PDF dans ChromaDB

```
def numero_page(chemin):
    correspondance = re.search(
      r"page_(\d+)", Path(chemin).stem)
    return int(correspondance.group(1)) if correspondance else -1
```

```
# PDF_PATH provient des cellules précédentes
PDF_PATH = Path(PDF_PATH)
# Identifiant lié au contenu du PDF
document_id = hashlib.sha256(PDF_PATH.read_bytes()).hexdigest()[:24]
# Supprimer l'ancienne version de l'index de ce PDF.
collection.delete(where={"document_id": document_id})
ids = []
documents = []
metadatas = []
```

PDF → identifiant de contenu → suppression de l'index associé → lecture des textes → découpage → embeddings CLIP → stockage dans ChromaDB

```
for fichier in sorted(TEXT_DIR.glob("page_*.txt")):
    texte = fichier.read_text(encoding="utf-8")
    page = numero_page(fichier)
    for index, fragment in enumerate(decouper_texte(texte)):
        ids.append(f"{document_id}:texte:{page}:{index}")
        documents.append(fragment)
        metadatas.append({
            "type": "texte", "document_id": document_id,
            "pdf": PDF_PATH.name, "page": page,
            "fragment": index,"source": str(fichier.resolve())
        })
# Indexation par lots
TAILLE_LOT = 64
for debut in range(0, len(documents), TAILLE_LOT):
    fin = debut + TAILLE_LOT
    lot = documents[debut:fin]
    collection.upsert(
        ids=ids[debut:fin],
        documents=lot,
        metadatas=metadatas[debut:fin],
        embeddings=encoder_textes(lot)
    )
print(f"{len(documents)} fragments de texte indexés.")
```

RAG Multi Modal

Indexation des images du PDF dans ChromaDB

```
extensions = {".png", ".jpg", ".jpeg", ".webp", ".bmp", ".tif", ".tiff", ".jp2"}
nombre_images_indexees = 0
for fichier in sorted(IMAGE_DIR.iterdir()):
    if not fichier.is_file() or fichier.suffix.lower() not in extensions:
        continue
    try:
        vecteur = encoder_image(fichier)

        collection.upsert(
            ids=[f"{document_id}:image:{fichier.name}"],
            embeddings=[vecteur],
            documents=[f"Image extraite du PDF : {fichier.name}"],
            metadatas=[{
                "type": "image",
                "document_id": document_id,
                "pdf": PDF_PATH.name,
                "page": numero_page(fichier),
                "source": str(fichier.resolve())
            }]
        )
        nombre_images_indexees += 1
    except Exception as erreur:
        print(f"Image ignorée : {fichier.name} — {erreur}")
print(f"{nombre_images_indexees} images indexées.")
print(f"Total dans ChromaDB : {collection.count()} éléments.")
```

Fichiers image →
filtrage →
embeddings CLIP →
enregistrement dans ChromaDB →
bilan

RAG Multi Modal

Recherche multimodale dans ChromaDB: Rechercher avec du texte ou une image

```
def rechercher(texte=None, image=None, type_resultat=None, k=5):
    if (texte is None) == (image is None):
        raise ValueError("Fournissez soit un texte, soit une image.")
    if type_resultat not in (None, "texte", "image"):
        raise ValueError("type_resultat doit être 'texte', 'image' ou None.")
    if k < 1:
        raise ValueError("k doit être supérieur ou égal à 1.")
    total = collection.count()
    if total == 0:
        raise ValueError("La collection est vide.")
    vecteur = ( encoder_textes([texte])[0] if texte is not None  else encoder_image(image) )
    options = {}
    if type_resultat is not None:
        options["where"] = {"type": type_resultat}
    return collection.query(
        query_embeddings=[vecteur],n_results=min(k, total), include=["documents", "metadatas", "distances"],
        **options
    )
```

Texte ou image →
embedding CLIP →
filtre facultatif →
recherche vectorielle →
résultats et métadonnées**

RAG Multi Modal

Affichage des résultats

```
from IPython.display import display
def afficher_resultats(resultats):
    for rang, (document, metadata, distance) in enumerate(
        zip( resultats["documents"][0], resultats["metadatas"][0], resultats["distances"][0]),
        start=1
    ):
        print( f"\n{rang}. {metadata['type'].upper()} | PDF : {metadata['pdf']} "
            f"| Page : {metadata['page']} | Similarité cosinus : {1 - distance:.3f}"
        )
        if metadata["type"] == "image":
            chemin = Path(metadata["source"])
            if chemin.is_file():
                with Image.open(chemin) as image:
                    apercu = image.convert("RGB")
                    apercu.thumbnail((600, 450))
                    display(apercu)
            else:
                print("Fichier image introuvable :", chemin)
        else:
            print(document)
```

```
resultats = rechercher(
    texte="a chart showing sales growth",
    k=5
)
afficher_resultats(resultats)
```

RAG Multi Modal

Affichage des résultats

```
resultats = rechercher(
    texte="a chart showing sales growth",
    k=5
)
afficher_resultats(resultats)
```

```
1. TEXTE | PDF : attention is all you need.pdf | Page : 2 | Similarité cosinus : 0.630
in section 3 . 2 . self - attention , sometimes called intra - attention is an attention mech

2. TEXTE | PDF : attention is all you need.pdf | Page : 2 | Similarité cosinus : 0.602
first is a multi - head self - attention mechanism , and the second is a simple , position -

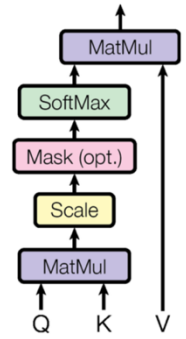
3. TEXTE | PDF : attention is all you need.pdf | Page : 3 | Similarité cosinus : 0.597
- product attention we call our particular attention " scaled dot - product attention " ( fig

4. TEXTE | PDF : attention is all you need.pdf | Page : 8 | Similarité cosinus : 0.595
), we vary the number of attention heads and the attention key and value dimensions , keeping

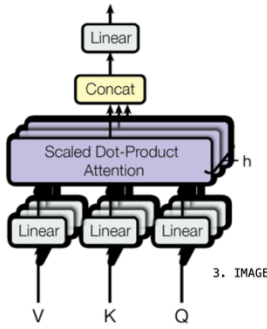
5. TEXTE | PDF : attention is all you need.pdf | Page : 7 | Similarité cosinus : 0.592
feed - forward layer , the approach we take in our model . as side benefit , self - attention
```

```
resultats = rechercher(
    texte="a photograph of a building",
    type_resultat="image",
    k=5
)
afficher_resultats(resultats)
```

1. IMAGE | PDF : attention is all you need.pdf | Page : 4 | Similarité cosinus : 0.140

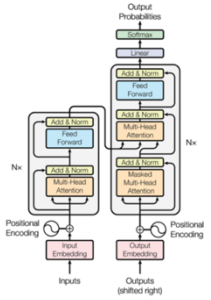


2. IMAGE | PDF : attention is all you need.pdf | Page : 4 | Similarité cosinus : 0.105




```
resultats = rechercher(
    image="./resultats_pdf/images/page_003_image_94.png",
    k=5
)
afficher_resultats(resultats)
```

3. IMAGE | PDF : attention is all you need.pdf | Page : 3 | Similarité cosinus : 0.084



Agentic AI



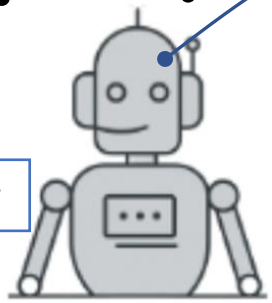
Quelle est l'origine de cette panne?



System message
Examples
Prompt
User Message
Goal
Memory



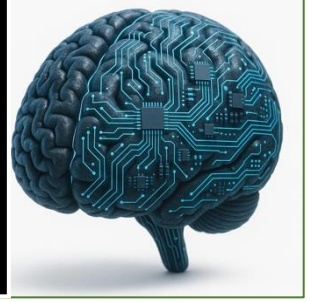
Agent = MIND + Body

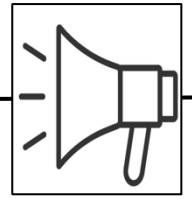


Agent

Reasoning P + Tools
Action
Reasoning P+ Obs +T
Action
Reasoning
Response
MIND

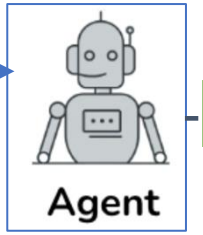
LLM





Réponse avec du texte et des images

A2A



Agent

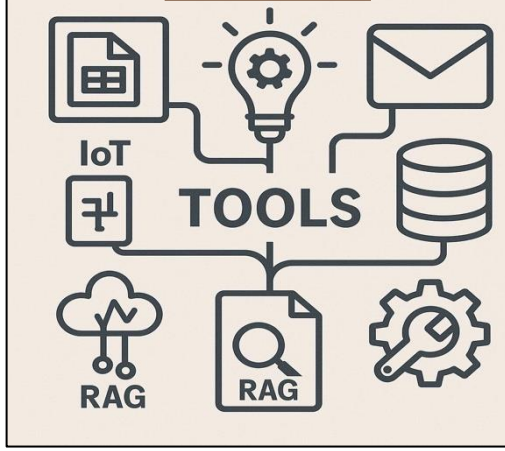
MCP

Tools
R4
R5
R6

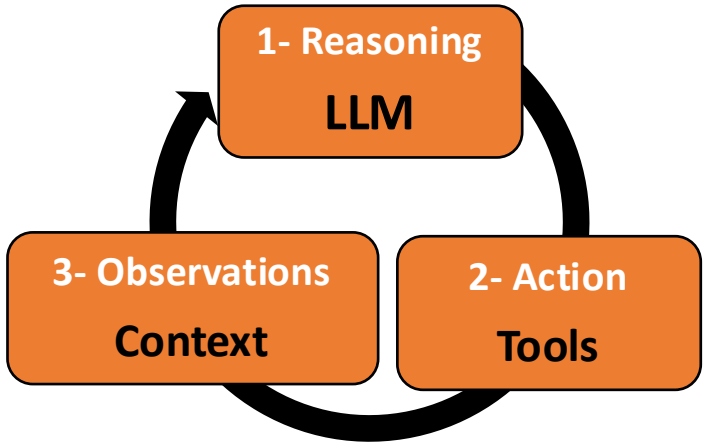
Action
observations

MCP

BODY



ReAct Pattern

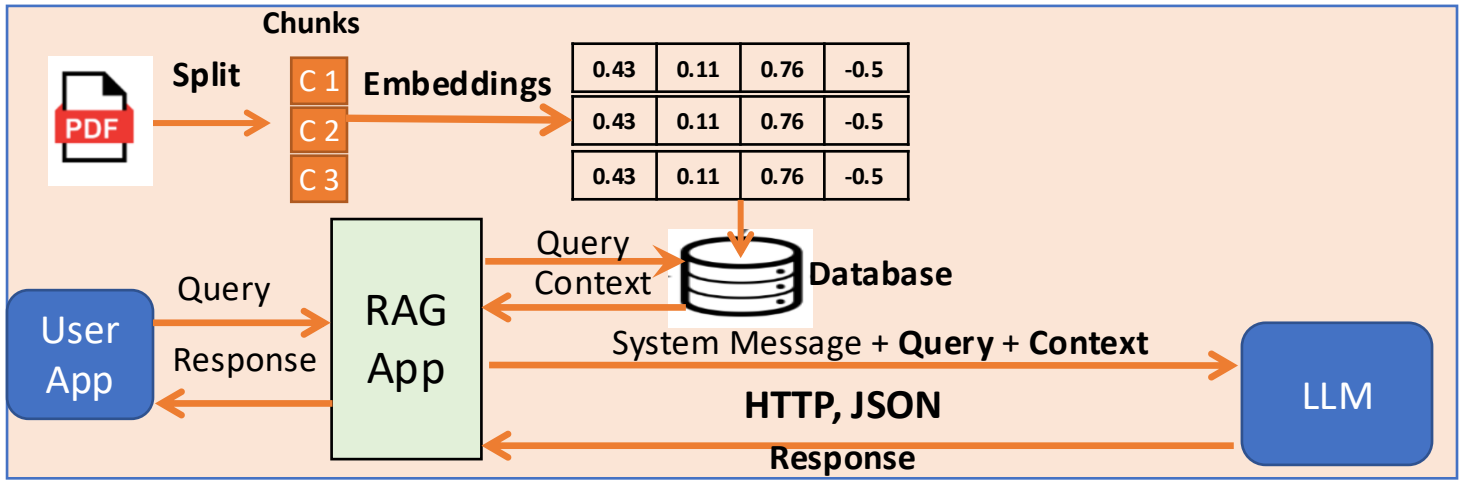


Frameworks an Agentic AI Platforms

- No Code Low Code :
  - N8N, Agent Builder, etc.
- High Level Frameworks
  - Langchain
  - Crew AI
  - Agents SDK
  - Phi Data
  - Auto Gen
  - Smol Agent
- Low Level Frameworks :
  - LangGraph
  - Pour Java : Spring AI, Langchain4J

Agentic RAG

Classic RAG



Agentic RAG

